

目录

## ——Anti-Interference-2D 目录

目录

Anti-Interference-2D 目录

目录

### 1. 目录

#### 1.1 目录 [30+]

目录

| 目录 | 目录 | 目录                  | 目录 |
|----|----|---------------------|----|
| 目录 | 目录 | $I(x) \cdot x_0$ 目录 | 目录 |
| 目录 | 目录 | $I(x) \cdot x_0$ 目录 | 目录 |
| 目录 | 目录 | $I(x) \cdot x_0$ 目录 | 目录 |
| 目录 | 目录 | $I(x) \cdot x_0$ 目录 | 目录 |

#### 1.2 目录 [30+]

目录 → 目录 → 目录 → 目录 → 目录 → 目录

目录

| 目录 | 目录     | 目录     | Anti-Interference-2D 目录 |
|----|--------|--------|-------------------------|
| 目录 | 目录     | 5×5 目录 | 目录                      |
| 目录 | Sobel  | 目录     | 目录                      |
| 目录 | 目录     | 目录     | 目录                      |
| 目录 | Th1Th2 | 目录     | 60-80% 30-50%           |
| 目录 | 8 目录   | 目录     | 目录                      |

## 2. 边缘检测

### 2.1 边缘检测[30分]

#### (1) Sobel

3×3卷积核

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

步骤

1. 计算  $G_x$  和  $G_y$
2. 计算  $G = \sqrt{G_x^2 + G_y^2}$
3. 计算  $\theta = \arctan(G_y / G_x)$

实现

```
import cv2
import numpy as np

def sobel_edge_detection(gray_image):
    """Sobel"""
    # 计算 Gx 和 Gy
    sobelx = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
    sobely = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)

    # 计算边缘强度
    magnitude = np.sqrt(sobelx**2 + sobely**2)
    magnitude = np.uint8(magnitude / magnitude.max() * 255)

    # 计算方向
    direction = np.arctan2(sobely, sobelx) * 180 / np.pi

    return magnitude, direction
```

结果

- 边缘检测
- 边缘检测
- 边缘检测

#### (2) Canny

实现步骤

| 步骤       | 操作       | 参数      | 说明                      |
|----------|----------|---------|-------------------------|
| 1. 高斯滤波  | 5×5卷积核   | 标准差     | $\sigma=1.0-2.0$        |
| 2. 非极大抑制 | Sobel    | 边缘强度    | -                       |
| 3. 双阈值检测 | 边缘强度     | 边缘强度    | -                       |
| 4. 边缘连接  | Th1, Th2 | Th1/Th2 | Th1=60-80%Th2=30-50%Th1 |
| 5. 边缘检测  | 8邻域      | 边缘强度    | -                       |

实现

```
def canny_edge_detection(gray_image, low_threshold=50, high_threshold=150):
    """Canny"""
    # 
    blurred = cv2.GaussianBlur(gray_image, (5, 5), 1.5)

    # Canny
    edges = cv2.Canny(blurred, low_threshold, high_threshold)

    return edges

def adaptive_canny(gray_image):
    """Canny"""
    # 
    sobelx = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
    sobely = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)
    magnitude = np.sqrt(sobelx**2 + sobely**2)

    # 
    high_threshold = np.percentile(magnitude, 85)
    low_threshold = high_threshold * 0.4

    # 
    blurred = cv2.GaussianBlur(gray_image, (5, 5), 1.5)
    edges = cv2.Canny(blurred, low_threshold, high_threshold)

    return edges
```

### Anti-Interference-2D

```
# Canny
canny_params = {
    'gaussian_ksize': 5,      # 
    'gaussian_sigma': 1.5,   # 
    'high_threshold_percentile': 85, # 
    'low_threshold_ratio': 0.4, # /
}
```

### (3) Laplacian [30†]

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

-

-

-

```
def laplacian_edge_detection(gray_image):
    """Laplacian"""
    # 
    blurred = cv2.GaussianBlur(gray_image, (3, 3), 0)

    # Laplacian
    laplacian = cv2.Laplacian(blurred, cv2.CV_64F)

    # 
    edges = cv2.convertScaleAbs(laplacian)

    return edges
```

## 2.2 [30+]

|  | - |  |
|--|---|--|
|  | - |  |
|  | - |  |

```
def morphological_edge_detection(gray_image, kernel_size=3):
    """"""
    kernel = np.ones((kernel_size, kernel_size), np.uint8)

    # 
    dilated = cv2.dilate(gray_image, kernel)
    eroded = cv2.erode(gray_image, kernel)

    # 
    gradient = cv2.absdiff(dilated, eroded)
    _, edges = cv2.threshold(gradient, 30, 255, cv2.THRESH_BINARY)

    return edges

def morphological_close(edges, kernel_size=3):
    """"""
    kernel = np.ones((kernel_size, kernel_size), np.uint8)
    closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel, iterations=2)
    return closed
```

## 2.3 (ACM)[48+]

-

-

-

$$E = E_{\text{int}} + E_{\text{ext}} = \int \left[ \alpha(s) \left| \frac{\partial C}{\partial s} \right|^2 + \beta(s) \frac{\partial^2 C}{\partial s^2} \right] ds + \int P(C(s)) ds$$

'''

- \$E\_{int}\$  
 - \$E\_{ext}\$  
 - \$\alpha, \beta\$

'''

```
def active_contour_segmentation(image, initial_mask, iterations=100):
    """
    # 
    contour = initial_mask.copy()

    # 
    for i in range(iterations):
        # 
        grad_x = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
        grad_y = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
        gradient = np.sqrt(grad_x**2 + grad_y**2)

        # 
        contour = contour + 0.1 * gradient * (image - contour)

        # 
        contour = cv2.GaussianBlur(contour.astype(np.float32), (3, 3), 0.5)

    return contour
```

## 2.4



## 3.

### 3.1

#### (1) ResNet-18

ResNet-18

```
ResNet-18
conv1 (7×7, 64) → maxpool (3×3)
├─ layer1 (3×3, 64×3) → 1/4
├─ layer2 (3×3, 128×4) → 1/8
├─ layer3 (3×3, 256×6) → 1/16
└─ layer4 (3×3, 512×3) → 1/32
```

-

- 
- 

```
import torch
import torch.nn as nn
from torchvision.models import resnet18

class ResNet18EdgeExtractor(nn.Module):
    """ResNet-18"""

    def __init__(self, pretrained=True):
        super().__init__()
        # ResNet-18
        resnet = resnet18(pretrained=pretrained)

        #
        self.conv1 = resnet.conv1
        self.bn1 = resnet.bn1
        self.relu = resnet.relu
        self.maxpool = resnet.maxpool

        self.layer1 = resnet.layer1 # 64
        self.layer2 = resnet.layer2 # 128 1/8
        self.layer3 = resnet.layer3 # 256 1/16

    def forward(self, x):
        #
        x1 = self.conv1(x) # 1/2
        x1 = self.bn1(x1)
        x1 = self.relu(x1)
        x1_pool = self.maxpool(x1) # 1/4

        x2 = self.layer1(x1_pool) # 1/4
        x3 = self.layer2(x2) # 1/8
        x4 = self.layer3(x3) # 1/16

        return [x2, x3, x4] #

    def extract_edge_features(self, x):
        """
        features = self.forward(x)

        # layer2
        edge_features = self.edge_head(features[1])

        return edge_features
```

## (2) Faster R-CNN [98]

(RPN)

|  | ResNet-50 |     |
|--|-----------|-----|
|  | RPN       | RoI |
|  |           |     |
|  | " " "     |     |

□□□□□□

```
class FasterRCNNEdgeDetector:
    """Faster R-CNN"""

    def __init__(self, model_path):
        self.model = torch.load(model_path)
        self.model.eval()

    @torch.no_grad()
    def detect_edges(self, image):
        """ """
        # 
        input_tensor = self.preprocess(image)

        # 
        predictions = self.model(input_tensor)

        # 
        edges = self.postprocess(predictions)

        return edges

    def preprocess(self, image):
        """ """
        #  tensor
        pass

    def postprocess(self, predictions):
        """ """
        #  NMS
        pass
```

### 3.2 〇〇〇〇〇〇〇〇〇〇〇〇

### (1) HED (Holistically-Nested Edge Detection)[28†]

□□□□□

The diagram illustrates the fusion of multiple 1D convolutions. It starts with a 1D input vector of size 4, represented by four small squares. This input is processed by five parallel 1D convolutional layers, each with a kernel size of 1. The layers are labeled Side1 (conv1\_2), Side2 (conv2\_2), Side3 (conv3\_3), Side4 (conv4\_3), and Side5 (conv5\_3). The output of each layer is scaled by a factor of 1/2, 1/4, 1/8, 1/16, and 1/32, respectively. The outputs of these five layers are then fused into a single operation, represented by a large square labeled 'Fuse' with a 1x1 kernel. The final output is a 1D vector of size 4, represented by four small squares.

111

- 
- 
- 

## (2) CASENet (Categorical Edge Detection)

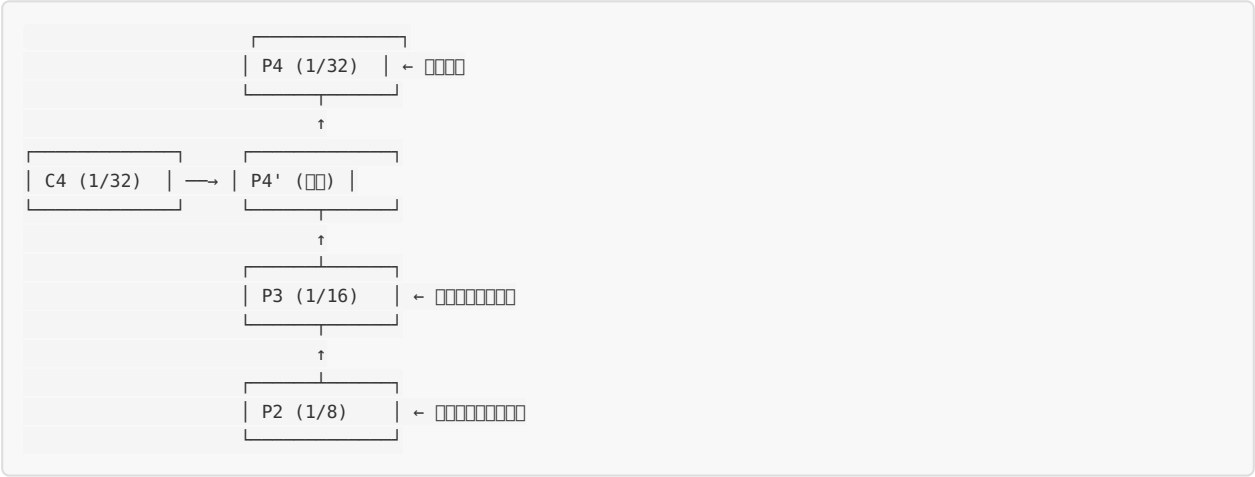




| FPN |  |  |
|-----|--|--|
|     |  |  |
|     |  |  |

4.

4.1 FPN[28+]



FPN

```

class FeaturePyramidNetwork(nn.Module):
    """FPN"""

    def __init__(self, in_channels_list=[256, 512, 1024, 2048]):
        super().__init__()

        # lateral convs
        self.lateral_convs = nn.ModuleList([
            nn.Conv2d(in_ch, 256, 1)
            for in_ch in in_channels_list
        ])

        # fpn convs
        self.fpn_convs = nn.ModuleList([
            nn.Conv2d(256, 256, 3, padding=1)
            for _ in in_channels_list
        ])

    def forward(self, features):
        # lateral convs
        laterals = [lateral_conv(features[i])
                    for i, lateral_conv in enumerate(self.lateral_convs)]

        # fpn convs
        for i in range(len(laterals) - 1, 0, -1):
            laterals[i-1] += F.interpolate(
                laterals[i], size=laterals[i-1].shape[-2:], mode='nearest')

        # outputs
        outputs = [fpn_conv(lat) for fpn_conv, lat
                   in zip(self.fpn_convs, laterals)]

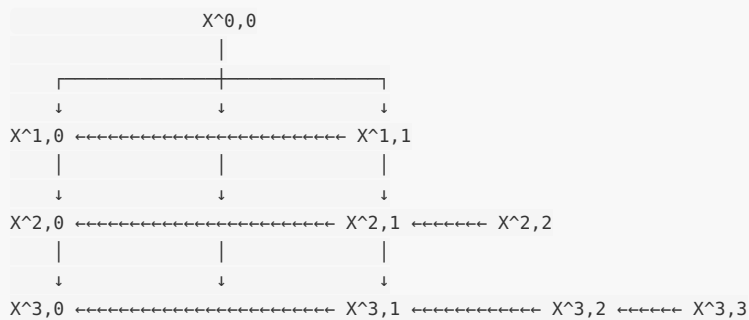
        return outputs

```

## 4.2 UNet++ [48†]

UNet++

UNet++



UNet++

- UNet++
- UNet++
- UNet++

4.3

| EdgeFPN       |  |  |
|---------------|--|--|
| FPN+attention |  |  |
| HRNet         |  |  |

5.

5.1

1.

2.

3.

4.

```

def region_growing(image, seeds, threshold=10):
    """
    区域生长

    Args:
        image: 图像
        seeds: 种子 [(x, y), ...]
        threshold: 阈值

    Returns:
        mask: 掩膜
    """
    h, w = image.shape
    mask = np.zeros((h, w), dtype=np.uint8)
    visited = np.zeros((h, w), dtype=bool)

    # 8邻域
    directions = [(-1, -1), (-1, 0), (-1, 1),
                  (0, -1), (0, 1),
                  (1, -1), (1, 0), (1, 1)]

    seed_value = image[seeds[0][1], seeds[0][0]]
    stack = list(seeds)

    while stack:
        x, y = stack.pop()

        if visited[y, x]:
            continue
        visited[y, x] = True

        # 遍历8邻域
        if abs(int(image[y, x]) - int(seed_value)) <= threshold:
            mask[y, x] = 255

        # 遍历8邻域
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < w and 0 <= ny < h and not visited[ny, nx]:
                stack.append((nx, ny))

    return mask

def auto_seed_selection(image):
    """
    自动种子选择
    """
    # 寻找局部最大值
    # 寻找高亮区域
    from scipy.ndimage import maximum_filter

    local_max = maximum_filter(image, size=20)
    seeds = np.where((image == local_max) & (image > np.percentile(image, 90)))

    return list(zip(seeds[1], seeds[0]))

```

区域生长掩膜

## 5.2 [30+]

| Step | Method       | Input | Anti-Interference-2D |
|------|--------------|-------|----------------------|
| 1    | Thresholding | Image | Binary               |
| 2    | Thresholding | Image | Binary               |
| 3    | Otsu         | Image | Binary               |
| 4    | Thresholding | Image | Binary               |

Code

```
def adaptive_threshold_segmentation(image, block_size=11, c=2):
    """
    Adaptive threshold segmentation
    Args:
        image: Image
        block_size: Block size
        c: Constant
    Returns:
        binary: Binary image
    """
    # Adaptive threshold
    binary = cv2.adaptiveThreshold(
        image, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        block_size,
        c
    )
    return binary

def otsu_threshold(image):
    """Otsu threshold"""
    _, binary = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    return binary

def multi_level_threshold(image, levels=3):
    """Multi-level threshold"""
    thresholds = []
    for i in range(1, levels):
        thresholds.append(np.percentile(image, 100 * i / levels))

    result = np.zeros_like(image)
    for i, th in enumerate(thresholds):
        result[image > th] = (i + 1) * (255 // levels)

    return result
```

## 5.3 [48+]

Code

Code

1. Image
2. Image 8-bit
3. Image/Video

4.

5.

```

def contour_tracking(binary_image):
    """
    Returns:
        contours: 
        hierarchy: 
    """
    contours, hierarchy = cv2.findContours(
        binary_image,
        cv2.RETR_EXTERNAL, # 
        cv2.CHAIN_APPROX_SIMPLE # 
    )

    return contours, hierarchy

def analyze_contour(contour):
    """
    # 
    area = cv2.contourArea(contour)

    # 
    perimeter = cv2.arcLength(contour, True)

    # 
    x, y, w, h = cv2.boundingRect(contour)

    # 
    rect = cv2.minAreaRect(contour)
    box = cv2.boxPoints(rect)
    box = np.int0(box)

    # 
    circularity = 4 * np.pi * area / (perimeter ** 2) if perimeter > 0 else 0

    # 
    compactness = perimeter ** 2 / area if area > 0 else 0

    return {
        'area': area,
        'perimeter': perimeter,
        'bbox': (x, y, w, h),
        'min_rect': rect,
        'circularity': circularity,
        'compactness': compactness
    }

```

## 5.4 [15+]

| Harris |  |  |
|--------|--|--|
| SIFT   |  |  |
| ORB    |  |  |
| AKAZE  |  |  |

5/5

```
def harris_corner_detection(gray_image, block_size=2, ksize=3, k=0.04):
    """Harris"""
    # 
    corners = cv2.cornerHarris(gray_image, block_size, ksize, k)

    # 
    threshold = 0.01 * corners.max()
    corner_points = np.where(corners > threshold)

    return corner_points

def sift_feature_detection(gray_image):
    """SIFT"""
    sift = cv2.SIFT_create()
    keypoints, descriptors = sift.detectAndCompute(gray_image, None)

    return keypoints, descriptors

def orb_feature_detection(gray_image, num_features=500):
    """ORB"""
    orb = cv2.ORB_create(nfeatures=num_features)
    keypoints, descriptors = orb.detectAndCompute(gray_image, None)

    return keypoints, descriptors
```

**6.** □□□□□□□□□□

## 6.1 環境問題

| 項目 | 内容   | 単位   |
|----|------|------|
| 1  | 基礎知識 | 10単位 |
| 2  | 基礎知識 | 10単位 |
| 3  | 基礎知識 | 10単位 |

□□□□□□□

```
def polarized_image_processing(rgb_image, polarization_degree):
    """
    极化图像处理

    Args:
        rgb_image: RGB图
        polarization_degree: 极化程度

    Returns:
        processed: 处理后的图
    """
    gray = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY)

    # 生成高亮掩码
    highlight_mask = (polarization_degree > 0.7).astype(np.float32)

    # 抑制高亮区域
    suppressed = gray.copy()
    suppressed = suppressed * (1 - 0.5 * highlight_mask)

    # CLAHE增强
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
    enhanced = clahe.apply(suppressed.astype(np.uint8))

    return enhanced
```

## 6.2 HDR

```
def hdr_edge_fusion(image_stack, exposure_times):
    """
    HDR边缘融合

    Args:
        image_stack: 图像堆栈
        exposure_times: 曝光时间

    Returns:
        fused_image: HDR融合后的图像
        edge_image: 边缘检测后的图像
    """
    # Mertens融合
    merge_mertens = cv2.createMergeMertens()
    fused = merge_mertens.process(image_stack)
    fused = np.clip(fused * 255, 0, 255).astype(np.uint8)

    # 边缘检测
    gray = cv2.cvtColor(fused, cv2.COLOR_RGB2GRAY)
    edges = cv2.Canny(gray, 50, 150)

    return fused, edges
```

## 6.3



```

def auto_canny_parameters(image):
    """
    Canny

    Returns:
        (low_threshold, high_threshold)
    """
    #
    sobelx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
    sobely = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
    magnitude = np.sqrt(sobelx**2 + sobely**2)

    #
    img_std = np.std(image)
    img_mean = np.mean(image)

    #
    high_threshold = np.percentile(magnitude, 85)
    low_threshold = high_threshold * 0.4

    #
    if img_std > 50:
        high_threshold *= 1.2
        low_threshold *= 1.2

    return int(low_threshold), int(high_threshold)

def steel_edge_detection_pipeline(image, polarization_info=None):
    """
    Anti-Interference-2D
    """
    # 1.
    if polarization_info is not None:
        processed = polarized_image_processing(image, polarization_info)
    else:
        processed = cv2.cvtColor(image, cv2.COLOR_RGB2GRAY)

    # 2.
    denoised = cv2.fastNlMeansDenoising(processed, h=10)

    # 3. Canny
    low_th, high_th = auto_canny_parameters(denoised)
    blurred = cv2.GaussianBlur(denoised, (5, 5), 1.5)
    edges = cv2.Canny(blurred, low_th, high_th)

    # 4.
    kernel = np.ones((3, 3), np.uint8)
    closed = cv2.morphologyEx(edges, cv2.MORPH_CLOSE, kernel, iterations=2)

    # 5.
    contours, _ = cv2.findContours(closed, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    mask = np.zeros_like(closed)
    for contour in contours:
        if cv2.arcLength(contour, True) > 20: #
            cv2.drawContours(mask, [contour], -1, 255, -1)

    return mask, closed

```

## 7. 7.1

### 7.1

0.1-0.5

```
def subpixel_edge_by_integration(edges, gray_image, window_size=5):
    """
    Args:
        edges:
        gray_image:
        window_size:

    Returns:
        subpixel_edges:
    """
    h, w = gray_image.shape
    subpixel_edges = []

    for y, x in edges:
        if y < window_size or y >= h - window_size or \
            x < window_size or x >= w - window_size:
            continue

        #
        window = gray_image[y-window_size:y+window_size+1,
                             x-window_size:x+window_size+1]

        #
        sum_g = np.sum(window)
        if sum_g == 0:
            continue

        y_coords, x_coords = np.mgrid[0:window.shape[0], 0:window.shape[1]]
        x_center = np.sum(window * x_coords) / sum_g - window_size
        y_center = np.sum(window * y_coords) / sum_g - window_size

        subpixel_edges.append((x + x_center, y + y_center))

    return subpixel_edges
```

## 7.2

```
def parabolic_fitting(edges, gray_image):
    """
    Args:
        edges:
        gray_image:

    Returns:
        refined_edges:
    """
    refined = []

    for y, x in edges:
        #
        #
        if y > 0 and y < gray_image.shape[0] - 1:
            i0 = int(gray_image[y-1, x])
            i1 = int(gray_image[y, x])
            i2 = int(gray_image[y+1, x])

            #
            if i0 + i2 - 2*i1 != 0:
                delta = 0.5 * (i0 - i2) / (i0 + i2 - 2*i1)
                refined_y = y + delta
                refined.append((x, refined_y))
            else:
                refined.append((x, y))
        else:
            refined.append((x, y))

    return refined
```

## 8.

### 8.1

|                  | (ODS) |  |  |  |
|------------------|-------|--|--|--|
| <b>Sobel</b>     |       |  |  |  |
| <b>Canny</b>     |       |  |  |  |
| <b>Laplacian</b> |       |  |  |  |
|                  |       |  |  |  |
| <b>ACM</b>       |       |  |  |  |
| <b>ResNet+</b>   |       |  |  |  |
| <b>HED</b>       |       |  |  |  |
| <b>PiDiNet</b>   |       |  |  |  |

## 8.2

| <b>Canny</b> |  | 1-2   |
|--------------|--|-------|
|              |  | <0.5  |
|              |  | <0.3  |
|              |  | 0.1mm |

## 8.3

|               | 1080p | FPS  |
|---------------|-------|------|
| Sobel         | ~5ms  | >200 |
| Canny         | ~15ms | ~65  |
|               | ~10ms | ~100 |
| PiDiNet (GPU) | ~20ms | ~50  |
| ResNet-18+    | ~50ms | ~20  |
| HED           | ~60ms | ~15  |

## 9. Anti-Interference-2D

### 9.1

```
# Anti-Interference-2D
EDGE_CONFIG = {
    #
    'denoise': {
        'method': 'fastNlMeansDenoising',
        'h': 10,
    },

    #
    'polarization': {
        'enabled': True,
        'highlight_threshold': 0.7,
        'suppress_ratio': 0.5,
    },

    #
    'edge_detection': {
        'method': 'adaptive_canny',
        'gaussian_ksize': 5,
        'gaussian_sigma': 1.5,
        'high_threshold_percentile': 85,
        'low_threshold_ratio': 0.4,
    },

    #
    'morphology': {
        'kernel_size': 3,
        'close_iterations': 2,
        'min_contour_length': 20,
    },

    #
    'subpixel': {
        'enabled': True,
        'method': 'parabolic_fitting',
        'window_size': 5,
    },

    #
    'output': {
        'save_edges': True,
        'save_contours': True,
        'save_subpixel': True,
    }
}
```

## 9.2 □□□□

```

class SteelEdgeDetector:
    """
    Anti-Interference-2D
    """

    def __init__(self, config=None):
        self.config = config or EDGE_CONFIG

    def detect(self, rgb_image, polarization_info=None):
        """
        Args:
            rgb_image: RGB (H, W, 3)
            polarization_info: (H, W)

        Returns:
            edges:
            contours:
            subpixel_edges:
        """
        # 1.
        gray = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2GRAY)

        # 2.
        if self.config['denoise']['method'] == 'fastNlMeansDenoising':
            denoised = cv2.fastNlMeansDenoising(
                gray, h=self.config['denoise']['h'])
        else:
            denoised = gray

        # 3.
        if self.config['polarization']['enabled'] and polarization_info is not None:
            denoised = self._polarized_processing(
                denoised, polarization_info)

        # 4.
        edges = self._edge_detection(denoised)

        # 5.
        edges = self._morphology_processing(edges)

        # 6.
        contours, _ = cv2.findContours(
            edges, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

        # 7.
        filtered_contours = self._filter_contours(contours)

        # 8.
        subpixel_edges = None
        if self.config['subpixel']['enabled']:
            subpixel_edges = self._subpixel_refinement(
                filtered_contours, gray)

        return edges, filtered_contours, subpixel_edges

    def _polarized_processing(self, gray, polarization_degree):
        """
        highlight_mask = (polarization_degree >
            self.config['polarization']['highlight_threshold']).astype(np.float32)

        suppressed = gray * (1 - self.config['polarization']['suppress_ratio'] * highlight_mask)

        clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
        enhanced = clahe.apply(suppressed.astype(np.uint8))

```

```

        return enhanced

    def _edge_detection(self, gray):
        """ """
        blurred = cv2.GaussianBlur(
            gray,
            (self.config['edge_detection']['gaussian_ksize'],) * 2,
            self.config['edge_detection']['gaussian_sigma']
        )

        #
        sobelx = cv2.Sobel(blurred, cv2.CV_64F, 1, 0, ksize=3)
        sobely = cv2.Sobel(blurred, cv2.CV_64F, 0, 1, ksize=3)
        magnitude = np.sqrt(sobelx**2 + sobely**2)

        high_th = np.percentile(magnitude,
                                self.config['edge_detection']['high_threshold_percentile'])
        low_th = high_th * self.config['edge_detection']['low_threshold_ratio']

        edges = cv2.Canny(blurred, int(low_th), int(high_th))

        return edges

    def _morphology_processing(self, edges):
        """ """
        kernel = np.ones(
            (self.config['morphology']['kernel_size'],) * 2,
            np.uint8
        )

        closed = cv2.morphologyEx(
            edges,
            cv2.MORPH_CLOSE,
            kernel,
            iterations=self.config['morphology']['close_iterations']
        )

        return closed

    def _filter_contours(self, contours):
        """ """
        filtered = []
        for contour in contours:
            if cv2.arcLength(contour, True) >= self.config['morphology']['min_contour_length']:
                filtered.append(contour)
        return filtered

    def _subpixel_refinement(self, contours, gray):
        """ """
        all_edges = []
        for contour in contours:
            for point in contour:
                all_edges.append((point[0][1], point[0][0])) # (y, x)

        if self.config['subpixel']['method'] == 'parabolic_fitting':
            refined = parabolic_fitting(all_edges, gray)
        else:
            refined = subpixel_edge_by_integration(
                all_edges, gray,
                self.config['subpixel']['window_size']
            )

        return refined

```



## 10.

### OpenCV

```
import cv2
import numpy as np

# Canny
edges = cv2.Canny(gray_image, low_threshold, high_threshold)

# Sobel
sobelx = cv2.Sobel(gray_image, cv2.CV_64F, 1, 0, ksize=3)
sobely = cv2.Sobel(gray_image, cv2.CV_64F, 0, 1, ksize=3)
magnitude = cv2.magnitude(sobelx, sobely)

# Laplacian
laplacian = cv2.Laplacian(gray_image, cv2.CV_64F)

#
kernel = np.ones((3, 3), np.uint8)
gradient = cv2.morphologyEx(gray_image, cv2.MORPH_GRADIENT, kernel)
```

|                     |            | GitHub                                                                              |
|---------------------|------------|-------------------------------------------------------------------------------------|
| OpenCV DNN          | ONNX       |                                                                                     |
| ONNXRuntime         |            | microsoft/onnxruntime                                                               |
| TensorRT            | NVIDIA GPU | nvidia/tensorrt                                                                     |
| Edge detection SOTA | SOTA       | <a href="https://github.com/xpdota/EdgeNIPS">https://github.com/xpdota/EdgeNIPS</a> |

v1.0

2026-04-27

Anti-Interference-2D